

Day 5, Session 1: GPU Architecture & PyCUDA Programming

Duration: 2 hours (theory + live demonstrations)

Prerequisites: Days 1, 2, 3, & 4 (Process parallelism, vectorization concepts)

Demos folder: [day5/demo/](#)

Table of Contents

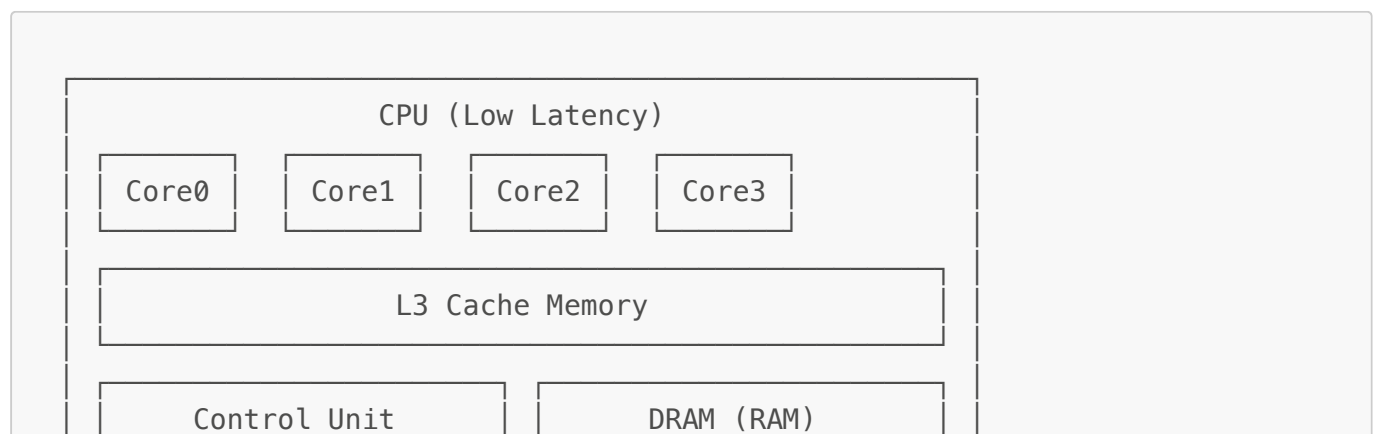
1. [Why GPUs for High-Performance Computing?](#)
2. [CPU vs GPU Hardware Architecture](#)
3. [The CUDA Execution & Thread Hierarchy](#)
 - [Grids, Blocks, Warps, and Threads](#)
 - [Memory Hierarchy: Registers, Shared Memory, and Global VRAM](#)
4. [PyCUDA Programming Model](#)
 - [The 5-Step CUDA Lifecycle](#)
 - [Writing CUDA C Kernels with `SourceModule`](#)
 - [Explicit Memory Management: `Alloc`, `Memcpy`](#)
5. [High-Level PyCUDA: `GPUArray` & `ElementwiseKernel`](#)
 - [NumPy on the GPU: `gpuarray.GPUArray`](#)
 - [Custom Fast Expressions: `ElementwiseKernel`](#)
 - [GPU MapReduce with `ReductionKernel`](#)
6. [Session 1 Summary](#)

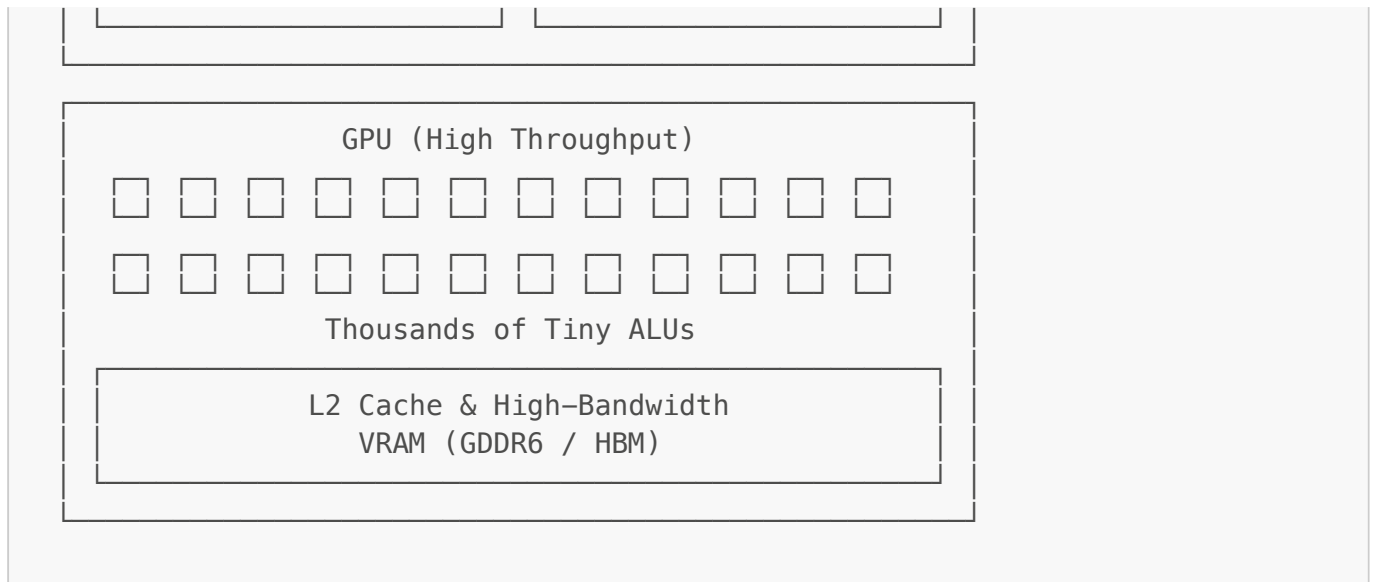
1. Why GPUs for High-Performance Computing?

Modern CPUs have 4 to 128 large, powerful cores optimized for **low-latency sequential execution**:

- Deep pipeline stages
- Massive caches (L1, L2, L3)
- Sophisticated branch predictors and speculative execution hardware

In contrast, GPUs (Graphics Processing Units) are massively parallel processors with **thousands of small, energy-efficient ALUs (Arithmetic Logic Units)** designed for **high-throughput compute**:





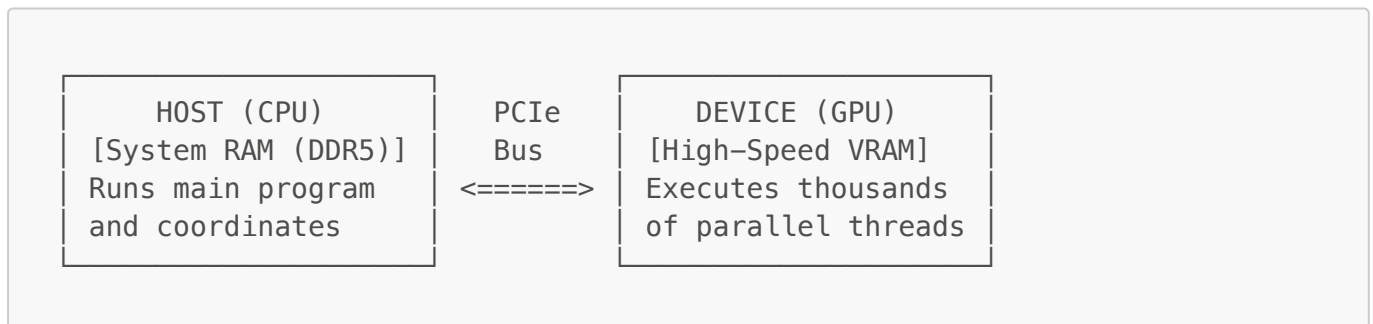
GPUs achieve **10x to 100x speedups** on data-parallel problems such as:

- Dense matrix multiplication & deep neural network training.
- Fast Fourier Transforms (FFT) and signal processing.
- Molecular dynamics, climate modeling, and finite element simulations.

2. CPU vs GPU Hardware Architecture

CUDA uses a **Heterogeneous (Hybrid) Computing Model**:

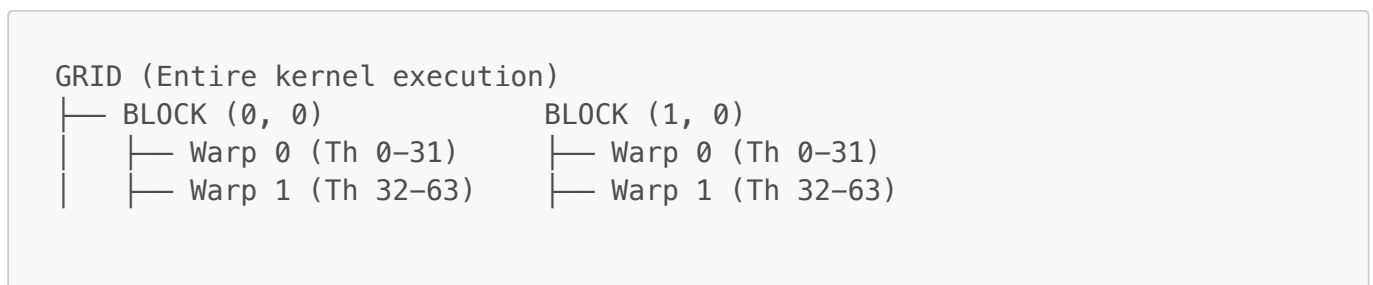
- **Host:** The CPU and its system RAM. Responsible for control flow, I/O, and launching kernels.
- **Device:** The GPU and its on-board Video RAM (VRAM). Responsible for parallel compute.

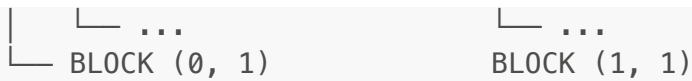


3. The CUDA Execution & Thread Hierarchy

Grids, Blocks, Warps, and Threads

CUDA organizes parallel execution into a three-tiered hierarchy:





1. **Thread:** The smallest unit of execution. Every thread executes the exact same kernel function on different data indices (**SIMT**: Single Instruction, Multiple Threads).
2. **Warp:** A hardware group of **32 consecutive threads** within a block that execute instructions simultaneously in lockstep.
3. **Thread Block:** A cooperative group of up to 1024 threads that run on the same Streaming Multiprocessor (SM) and can synchronize and share on-chip cache.
4. **Grid:** A collection of thread blocks launched across the entire GPU chip.

Calculating Global Thread Index

Each thread calculates its unique position in the dataset using built-in CUDA variables:

In 1D arrays:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

In 2D matrices (images, grids):

```
int col = blockIdx.x * blockDim.x + threadIdx.x; // X dimension -> Column
int row = blockIdx.y * blockDim.y + threadIdx.y; // Y dimension -> Row
int global_index = row * width + col;           // Flattened 1D array
index
```

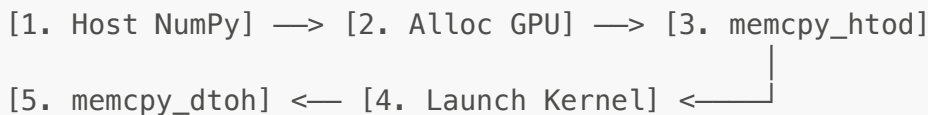
Memory Hierarchy

Memory Type	Location	Scope	Speed / Latency
Registers	On-chip	Single thread	Fastest (< 1 cycle)
Shared Memory	On-chip SRAM	Thread Block	Extremely fast (~1-2 cycles)
Constant Memory	On-chip cache	All threads (read-only)	Fast when cached
Global Memory (VRAM)	Off-chip GDDR/HBM	All threads & Host	Slower (200-400 cycles)

4. PyCUDA Programming Model

PyCUDA brings Python's elegance and rapid prototyping to NVIDIA's CUDA C runtime.

The 5-Step CUDA Lifecycle



1. Create input data in CPU memory (typically a NumPy array).
2. Allocate device memory on the GPU using `cuda.mem_alloc(nbytes)`.
3. Copy data from Host to Device via `cuda.memcpy_htod(gpu_buf, host_buf)`.
4. Compile and launch the CUDA C kernel with defined `block` and `grid` dimensions.
5. Copy the result back from Device to Host via `cuda.memcpy_dtoh(host_buf, gpu_buf)`.

Writing CUDA C Kernels with `SourceModule`

```

import pycuda.driver as cuda
import pycuda.autoinit
from pycuda.compiler import SourceModule
import numpy as np

# Write CUDA C kernel directly in a Python string:
cuda_code = """
__global__ void vector_add(float *c, const float *a, const float *b, int n)
{
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}
"""

mod = SourceModule(cuda_code)
vector_add = mod.get_function("vector_add")

# Launch on GPU: 256 threads per block
vector_add(c_gpu, a_gpu, b_gpu, np.int32(1024),
           block=(256, 1, 1), grid=(4, 1))
  
```

Demo Time

Now, let's see these theoretical topics in action.

- `01_pycuda_kernel_and_memory.py` —
 1. `demo_cuda_kernel_and_memory()` — Vector addition with explicit GPU memory allocation & transfers
 2. `demo_matrix_manipulation()` — 2D thread indexing for matrix operations

5. High-Level PyCUDA: GPUArray & ElementwiseKernel

Writing manual memory allocation and pointer math for simple array operations can be tedious. PyCUDA provides convenient high-level wrappers:

NumPy on the GPU: `gpuarray.GPUArray`

`GPUArray` provides an array interface identical to NumPy, but stored entirely in GPU VRAM:

```
import pycuda.gpuarray as gpuarray
import numpy as np

a_host = np.array([1, 2, 3, 4], dtype=np.float32)

# Copy to GPU in one step:
a_gpu = gpuarray.to_gpu(a_host)

# Arithmetic expressions execute directly as parallel GPU kernels!
b_gpu = 2.0 * a_gpu + 10.0

# Fetch result back to CPU:
b_host = b_gpu.get()
print(b_host) # [12., 14., 16., 18.]
```

Custom Fast Expressions: `ElementwiseKernel`

`ElementwiseKernel` compiles custom C expressions on-the-fly without requiring thread loop boilerplate:

```
from pycuda.elementwise import ElementwiseKernel

# Linear combination: z = a * x + y
saxpy = ElementwiseKernel(
    "float a, float *x, float *y, float *z",
    "z[i] = a * x[i] + y[i]",
    "saxpy_kernel"
)

saxpy(2.5, x_gpu, y_gpu, z_gpu)
```

GPU MapReduce with `ReductionKernel`

Calculates dot products, norms, sums, and min/max operations using tree reductions on the GPU:

```
from pycuda.reduction import ReductionKernel
```

```
# Calculate vector dot-product: sum(x[i] * y[i])
dot_product = ReductionKernel(
    np.float32,
    neutral="0",
    reduce_expr="a + b",      # Reduction step
    map_expr="x[i] * y[i]",   # Map step
    arguments="float *x, float *y"
)

result = dot_product(x_gpu, y_gpu).get()
```

Demo Time

Now, let's see these theoretical topics in action.

- `02_pycuda_gpuarray_elementwise.py` —
 1. `demo_gpuarray_vector_ops()` — NumPy-style vector arithmetic running in parallel on GPU
 2. `demo_elementwise_kernel()` — High-performance inline custom expressions
 3. `demo_map_reduce_gpu()` — GPU-accelerated parallel tree reduction

Session 1 Summary

Abstraction Level	Tool	How it Works
Low-Level CUDA C	<code>SourceModule</code>	Direct C kernel code, explicit memory allocation (<code>mem_alloc</code> , <code>memcpy</code>)
Mid-Level Expression	<code>ElementwiseKernel</code>	Generates CUDA kernel from single-line C expression
High-Level Array	<code>gpuarray.GPUArray</code>	Object-oriented NumPy syntax executed directly on GPU
MapReduce	<code>ReductionKernel</code>	Parallel map followed by tree reduction

Break time! Next up is Session 2: JIT Compilation with Numba & OpenCL with PyOpenCL → [README-session2.md](#)